Load-Intensive FastAPI Stress Platform on Kubernetes: Infrastructure Design and Crash-Prevention Through Vertical Scaling

Stress-Test Project

March 3, 2026

Live test https://stresstest.kristijanboshev.com/

Abstract

This paper analyzes a production-oriented CPU stress-testing platform implemented in Python/FastAPI and deployed on Kubernetes with Vertical Pod Autoscaler (VPA). The platform is intentionally designed to generate sustained high computational pressure through parallel algorithms, while preserving service availability and preventing process or pod crashes. The study focuses on three dimensions: (1) load generation characteristics at algorithm and runtime level, (2) infrastructure controls for health and resource isolation, and (3) scaling behavior under increasing demand. Results from architecture and code inspection show that stability is achieved through a layered strategy: admission control at API level, bounded resource envelopes in Kubernetes, adaptive CPU/memory rightsizing through VPA, and active health supervision via liveness/readiness probes. This design does not eliminate performance degradation under extreme pressure, but it prevents uncontrolled failure modes by converting overload into managed slowdowns and orchestrator-mediated recovery.

1 Introduction

Modern backend systems often fail not because average load is high, but because peak demand exceeds assumptions encoded in runtime configuration and infrastructure limits. CPU-bound services are especially vulnerable: they saturate quickly, can starve event loops, and may trigger container restarts when memory pressure rises.

The analyzed system addresses this challenge with a deliberate stress-first architecture. It exposes a web/API interface for triggering several computationally expensive workloads (prime generation, matrix multiplication, Fibonacci computations, parallel sorting, and Monte Carlo simulation). The same application also reports system telemetry and supports historical run analysis.

The central research question is: *How can a CPU-intensive microservice maintain operational*

stability under aggressive synthetic load, and to what extent does vertical scaling prevent crash conditions?

2 System Overview

The application stack contains four principal layers:

1. **Presentation and API layer:** FastAPI endpoints plus a dashboard UI for test orchestration and live metrics.
2. **Load engine:** A stress module implementing CPU-heavy algorithms with Python process pools across available cores.
3. **Monitoring layer:** A continuous collector (`psutil`) capturing CPU utilization, frequency, memory pressure, and optional temperature.
4. **Kubernetes control plane integration:** Deployment, Service, Ingress, and VPA manifests governing runtime health and resources.

At runtime, a stress test request transitions through endpoint validation, background execution, metrics collection, result persistence, and status publication via REST/WebSocket.

3 Load Model and Computational Characteristics

3.1 Workload Types

The stress engine uses heterogeneous CPU-intensive kernels:

- **Prime generation:** segmented sieve over integer ranges.
- **Matrix multiplication:** dense linear algebra with random matrices.
- **Fibonacci sequence:** repeated iterative large-integer progression.
- **Parallel merge sort:** recursive decomposition with multiprocessing at shallow depths.
- **Monte Carlo π:** embarrassingly parallel random sampling.

This diversity is important: not all CPU loads stress the same bottleneck. Some are compute-arithmetic dominant, others include significant memory bandwidth and allocation overhead.

3.2 Parallel Execution Strategy

The platform derives worker count from logical CPUs and dispatches chunked work via process pools. In simplified form, end-to-end runtime can be approximated by:

$$T_{total} \approx \max_i(T_{chunk,i}) + T_{startup} + T_{merge} + T_{ipc}$$

where T_{ipc} captures inter-process communication and result aggregation overhead. As worker count increases, ideal speedup is limited by sequential fractions and overhead, consistent with Amdahl-type behavior:

$$S(n) = \frac{1}{(1-p) + \frac{p}{n} + \delta(n)}$$

with p as parallelizable fraction and $\delta(n)$ overhead growth.

3.3 Admission Control as Load Safety

The API enforces single-test concurrency through a global running-state guard. If a test is active, additional test-trigger requests return an error rather than enqueuing unbounded work. This is a critical anti-crash mechanism: it transforms accidental concurrent CPU storms into explicit back-pressure.

4 Infrastructure Architecture on Kubernetes

4.1 Deployment Topology

The service runs in a dedicated namespace (`stress-test`) with:

- **Deployment** (single replica),
- **ClusterIP Service** exposing port 8000 internally,
- **Ingress** via Traefik with TLS entrypoint,
- **VPA** controlling CPU and memory requests/limits.

4.2 Container Runtime Boundaries

The deployment defines:

- requests: `500m CPU`, `1Gi memory`
- limits: `8000m CPU`, `16Gi memory`

These values create a bounded operating envelope. The lower bound supports startup and baseline responsiveness; the upper bound prevents node-level resource monopolization and catastrophic noisy-neighbor effects.

4.3 Health Supervision

Both liveness and readiness probes target `/health`. Their role differs:

- **Readiness probe:** removes overloaded/unready pods from traffic.
- **Liveness probe:** restarts pods that become non-functional.

This dual-probe strategy prevents prolonged black-hole behavior where a pod exists but cannot safely serve requests.

5 Vertical Autoscaling Strategy

5.1 Why Vertical Instead of Horizontal

The deployment intentionally uses one replica and VPA, matching workload properties:

- stress runs are CPU/memory heavy inside a single process tree,
- state and history are in-process memory,
- admission policy allows one active test at a time.

Under this design, scaling *up* one pod is more direct than scaling *out* many pods.

5.2 VPA Policy Interpretation

VPA controls CPU and memory with:

- minimum allowed: `500m CPU`, `1Gi`
- maximum allowed: `8000m CPU`, `16Gi`
- mode: `Auto`
- update mode: `InPlaceOrRecreate`

Operationally, this means the pod can receive rightsized resources as observed demand grows, either in place or by controlled restart if required by platform constraints.

5.3 Crash-Prevention Mechanics of VPA

VPA contributes to crash prevention in three concrete ways:

1. **CPU starvation mitigation:** raising CPU allocation reduces event-loop starvation and long tail latencies.
2. **Memory pressure mitigation:** higher memory ceilings reduce OOM risks during matrix-heavy or large-array runs.
3. **Demand adaptation:** automatic adjustment minimizes prolonged mismatch between requests and actual usage.

Important limitation: VPA does not make the workload infinite-capacity. It prevents many crash classes by resizing within limits, but sustained demand beyond `8 CPU/16Gi` still requires architectural changes.

6 Observability and Experimental Methodology

6.1 Telemetry Signals

The monitor collects periodic samples (0.5–1.0s cadence depending on path) of:

- CPU utilization (aggregate and per-core snapshots),
- CPU frequency,
- memory percentage and absolute usage,
- optional thermal readings,
- test duration and peak values.

The system persists up to 1000 recent samples and records peak/average metrics per test execution.

6.2 Test Method

An infrastructure-focused evaluation should use escalating phases:

1. **Baseline:** no stress test active; collect idle utilization and response times.
2. **Single-test peak:** run each algorithm with default then high parameters.
3. **Comprehensive sequence:** execute all stress tests serially at elevated sizes.

4. **Recovery:** measure readiness restoration and post-load stability.

The primary dependent variables are crash incidence, restart count, failed probes, and sustained API responsiveness.

7 Results and Analysis

7.1 Observed Throughput Profile

Representative benchmark values in project documentation indicate high computational throughput, including millions of operations per second for suitable kernels, with substantial CPU utilization. This confirms the platform creates node-visible pressure rather than synthetic no-op load.

7.2 Failure Modes Without Scaling Controls

Without the current controls, expected failure modes include:

- concurrent test storms from repeated API calls,
- memory spikes from large matrix/array allocations,
- pod unavailability due to non-responsive process state,
- unstable latency due to CPU throttling or starvation.

7.3 How Scaling Prevents Crashing

The present architecture reduces failure probability through layered defense:

Control Layer	**Stability Contribution**
API admission guard	Prevents overlapping stress runs and unbounded concurrency.
Kubernetes resource bounds	Ensures workloads remain inside predictable CPU/memory envelopes.
VPA auto-rightsizing	Adapts requests/limits toward observed demand; reduces starvation and OOM risk.
Readiness probe	Removes unhealthy instances from traffic before total failure is visible to users.
Liveness probe	Triggers recovery restart for stuck or degraded process states.
Ingress + Service isolation	Maintains controlled routing and avoids direct pod exposure.

The key insight is that crash prevention here is not a single autoscaler feature; it is the composition of policy, orchestration, and runtime safeguards.

8 Discussion

8.1 Trade-offs of Single-Replica VPA Design

The chosen model prioritizes deterministic stress behavior and simpler operations, but introduces trade-offs:

- during recreate-style VPA updates, temporary interruption is possible,
- throughput remains bounded by one pod's vertical ceiling,
- in-memory history is not durable across restarts.

For a stress lab or controlled benchmarking service, these trade-offs are often acceptable.

8.2 When Horizontal Scaling Becomes Necessary

If requirements evolve toward multi-tenant concurrency or strict high availability, architecture should move to:

- stateless workers,
- externalized result storage,
- queue-based job admission,
- optional HPA+VPA coordination strategy.

9 Conclusion

This study shows that a CPU-stressing FastAPI application can remain operational under heavy load when scaling and reliability controls are designed as an integrated system. Vertical autoscaling is effective for preventing common crash modes (CPU starvation, memory exhaustion, unstable long-running pressure) when bounded by explicit policies and supported by health probes.

The most important architectural principle is *controlled pressure*: generate real load, but constrain and observe it at every layer. In this project, that principle is realized through one-test-at-a-time admission control, VPA-managed resource adaptation, strict CPU/memory boundaries, and orchestrator health checks. As a result, overload events are more likely to degrade gracefully than terminate catastrophically.